

Purana `var`

| Ye sirf purane code samajhne ke liye hai. Naye code me `let` / `const` use karo.

`var` aur `let` me 3 bade farak

1. `var` ka block scope nahi hota

`var` sirf **function** ya **global** scope maanta hai. `if` / `for` ke `{ }` ko ignore karta hai.

```
javascript

if (true) {
  var test = 5;
}
console.log(test); // 5 (bahar bhi chal gaya)

for (var i = 0; i < 3; i++) {}
console.log(i); // 3
```

Function ke andar `var` us function tak hi limited rehta hai.

2. `var` dobara declare ho sakta hai

```
javascript

var user = "Pete";
var user = "John"; // koi error nahi
```

`let` me same scope me dobara declare karoge to error aata hai.

3. `var` Hoisting

`var` ki **declaration** function ke start me upar chali jaati hai. Lekin **value assign** wahin hoti hai jahan likha hai.

```
javascript

function sayHi() {
  console.log(phrase); // undefined (error nahi)
  var phrase = "Hello";
}
```

JS ise aise samajhta hai:

```
javascript
```

```
var phrase;           // declaration upar
console.log(phrase);  // undefined
phrase = "Hello";     // assignment apni jagah
```

IIFE (Immediately Invoked Function Expression)

Pehle block scope nahi tha, to log function banakar turant call kar dete the taaki variables private rahein.

```
javascript

(function () {
  var message = "Hello";
  console.log(message);
})();
```

Function ko `( )` me isliye daalte hain taaki JS use Function Expression samjhe, Declaration nahi. Aaj iski zarurat nahi, purane code me dikh sakta hai.

Summary

Baat	<code>var</code>	<code>let/const</code>
Scope	function / global	block
Redeclare	ho jaata hai	error
Hoisting	upar chala jaata hai (value <code>undefined</code>)	dead zone, error

Isliye `let/const` hi use karo.